ACRME Calculation Logic — Plain English Walkthrough (v2.2)

> **Document type:** Companion reference — plain-language explanation of every formula in the Calculation Logic Reference (v2.2). This document does **not** replace the Calculation Logic Reference or the Technical Design Document. Read it alongside those documents to understand the *intent* behind each formula before reading the formal notation.
>
> **Audience:** Platform engineers, product owners, and architects who need to reason about ACRME behaviour without working through mathematical notation first.
>
> **Version:** 1.0 — 2 September 2026. Authored from the ACRME Q&A design walkthrough session.

Document Control

Field	Value
Title	ACRME Calculation Logic — Plain English Walkthrough
Version	1.0
Status	Baseline
Date	2 September 2026
Source	ACRME Q&A design walkthrough session; Calculation Logic Reference v2.2; Requirements Baseline v2.2
Companion documents	`acrme_calculation_logic_reference.md` (formal notation); `acrme_technical_design_document.md` (component design)

Table of Contents

---

1. Orientation — What ACRME Actually Does

Before any Azure VM is deployed for a customer, ACRME answers three questions in sequence:

1. **Where should this customer live?** — pick the best Azure region for each environment (production, CVAL, DR).
2. **Is there enough quota there?** — ensure the subscription has headroom to deploy the required VM family.
3. **Is there a physical reservation?** — ensure Azure has set aside the physical capacity the VM needs.

If all three answers are "yes," the deployment gets a `READY` signal and proceeds. If any answer is "no," the deployment is held until the gap is closed.

These three questions map to the three calculation domains this document explains:

Domain	Question answered	Core scenarios
Region Selection	Where does this customer go?	Scenarios 1, 2, 3, 4, 5, 6, 7
Quota Management	Is the subscription limit high enough?	Scenarios 8, 9, 10
Capacity Management	Has Azure reserved the physical hardware?	Scenarios 11, 12, 13, 16, 17, 18, 19

---

2. How This Document Is Organised

Each domain section follows the same pattern:

- **Concept** — what this domain manages, in plain language.
- **The formula, translated** — what each piece of the formula means.
- **Happy path** — what happens when everything is fine.
- **Sad path** — what happens when something is wrong.

Geography-specific examples use the authoritative Standard Capacity region sets (Requirements Baseline v2.2 Section 6, ADR-001):

Geography	Standard Capacity Regions (engine-selectable)	Restricted Regions (exception only)
US	West US 3 · Central US · Canada Central	East US 2
EU	Switzerland North · Sweden Central	North Europe · West Europe
Middle East	UAE North · Saudi Arabia Central	—

Representative SKU examples throughout: **E16ads_v5** (16 vCPU per VM) and **E8ads_v5** (8 vCPU per VM).

---

3. Domain 1 — Region Selection

3.1 Concept

Region selection answers the question: *"Of all the Azure regions available for this geography, which one should this customer's production (or CVAL, or DR) environment live in?"*

The engine scores every eligible region and picks the highest-scoring one. The score is a number between 0 and 1 — higher is better. The region with the highest score wins.

Selection happens in a fixed sequential order: **Prod first, then CVAL (Non-Prod), then DR**.

- The **Prod region** is usually supplied by the customer. ACRME validates it against hard constraints rather than deriving it. Deriving a Prod region from geography alone is an exception path that requires explicit approval.
- The **CVAL region** is selected by the engine from the remaining Standard regions after Prod is fixed.
- The **DR region** is selected last from the remaining Standard regions.

If a customer's supplied region is a **Restricted Capacity Region** (e.g. East US 2, North Europe), it does not enter scoring at all. It goes to a manual exception workflow.

3.2 The Hard Constraint Gate — Ruling Out Ineligible Regions

Before any scoring happens, each candidate region must pass a set of hard constraints (HC-1..HC-10). Think of these as eligibility checks that must all pass before a region is even considered. A region that fails any hard constraint is removed from the scoring candidate pool entirely.

The three arithmetic constraints that matter most in practice:

HC-3 (Quota Floor): Does the region have enough quota to accept the new deployment?

```
For Prod:   the region's Prod quota headroom   the vCPUs needed for this deployment
For NonProd: the region's effective NonProd headroom   the vCPUs needed
For DR:      the region's DR headroom   the target DR quantity × vCPU
```

In plain English: the region's subscription quota limit is high enough that adding these VMs will not push usage over the cap.

HC-6 (DR Coverage Floor): Does the region have enough room in its DR and NonProd CRGs combined to take on this customer's DR requirement?

HC-7 (DR Floor Integrity): Would adding these NonProd VMs eat into the space that is earmarked for DR? If yes, the region is rejected — ACRME never lets NonProd growth erode DR protection.

What happens if a region fails? It is excluded from the scoring step. The engine scores only the regions that passed all hard constraints.

3.3 The Scoring Formulas — What Each Weight Means

Each of the three environment types has its own scoring formula. All three share the same five weights (=0.30, =0.20, =0.25, =0.15, =0.10) but each weight measures something slightly different depending on the environment type.

Every component value is **clamped to the range [0, 1]** before being multiplied by its weight. This means a component can never go below 0 or above 1, regardless of the raw numbers.

Why These Specific Weights? The Building Analogy Before diving into the technical formulas, here is the plain-English intuition behind each weight, using the analogy of placing desks in office buildings:

Weight	Value	Plain English reason (building/office analogy)
= 0.30 (largest)	Headroom/capacity signal	**The single most important factor.** If a building is nearly full, we cannot grow — and growing is inevitable. Deserves the highest priority.

Weight	Value	Plain English reason (building/office analogy)
= **0.25**	Distribution fairness	We never want all customers in the same building. If one building catches fire (fails), we want as few customers affected as possible. **Second highest.**
= **0.20**	Quota headroom	The building permit (quota) must have room approved by the authority (Microsoft). Without it, no new desks can be added even if the floor is empty.
= **0.15**	DR readiness	Is the backup office already covering enough customers? A well-covered DR region is a better destination. Lower priority because DR coverage depends on the customer mix, not just space.
= **0.10** (smallest)	Zone diversity	Are the desks spread across different floors of the building? If one floor has a power cut, other floors keep running. Nice-to-have, not critical.

All five add to exactly 1.0 (100%). They are **policy defaults stored in config** — not hardcoded. The team can adjust them if priorities change (see Section 6 for the weight-sum validation requirement).

Component (weight 0.30) — Headroom or Capacity Signal This is the single biggest factor. It answers: *"How much free capacity does this region have?"*

Formula type	What it measures
PS_Prod	NonProd CRG effective free slots ÷ Prod CRG quantity (how much NonProd breathing room backs the Prod anchor)
PS_NonProd	NonProd CRG effective free slots ÷ NonProd CRG quantity (direct NonProd headroom ratio)
PS_DR	DR CRG free slots ÷ DR CRG quantity (direct DR headroom ratio)

In plain English: a region with more available headroom scores higher. This is the most heavily weighted factor because placing a customer in a region with no headroom creates an immediate risk of capacity exhaustion.

Component (weight 0.20) — Quota Headroom Signal Answers: *"How much room does the subscription's quota limit have?"*

All three environment types measure this the same way: the ratio of remaining quota headroom to total quota limit. A region where the subscription is nearly at its quota limit scores lower. This matters because even if Azure has physical capacity, a quota limit prevents deployment.

Component (weight 0.25) — Distribution Fairness Answers: *"Is this region already heavily loaded with customers?"*

All three environment types measure this as `1 - (customers already in this region ÷ total customers across all regions)`. The subtraction from 1 means a lightly loaded region scores high (close to 1) and a heavily loaded region scores low (close to 0).

In plain English: ACRME tries to spread customers evenly. A region that already hosts most of the customer base is less attractive for the next customer — even if it has adequate headroom.

Component (weight 0.15) — DR Readiness or Overflow Integrity Signal Answers: *"Is this region's DR position healthy?"*

Formula type	What it measures
PS_Prod	DR CRG coverage ratio (does this region have adequate DR coverage already?)

Formula type	What it measures
PS_NonProd	NonProd CRG effective free ÷ quantity (overflow capacity health — duplicate of in design-of-record)
PS_DR	DR CRG coverage ratio ÷ dr_ratio_target (how close is DR coverage to the bootstrap target?)

In plain English: this component rewards regions where the DR position is healthy — the region is not already overcommitted from a DR perspective.

Component (weight 0.10) — Zone Diversity Answers: *"How many Availability Zones does this region have?"*

The formula is `az_count ÷ 3`, clamped to [0, 1]. A region with 3 AZs scores 1.0; a region with 1 AZ scores 0.33. Zone diversity matters because it determines whether the deployment can be spread across multiple physical failure domains.

In plain English: a region with more Availability Zones is more resilient and scores higher on this component, all else equal.

3.4 Putting It Together — The Score and the Winner

Each region gets a score between 0 and 1:

```
Score(region) = 0.30 ×  _component
              + 0.20 ×  _component
              + 0.25 ×  _component
              + 0.15 ×  _component
              + 0.10 ×  _component
```

The region with the **highest score wins** — this is what the formula `argmax(PS)` means. If two regions tie, the engine picks the first one in the Standard Capacity Region list for that geography (deterministic tie-break).

The winning region's score, along with every input value and the PlacementPolicy version, is written to the OperationRecord so the decision can be replayed or audited later.

3.5 US Geography Walkthrough (3 Standard Regions)

Context: US Geography has three Standard Capacity regions: West US 3, Central US, Canada Central. All three are initially in the scoring candidate pool.

Happy Path — All Three Regions Eligible All three regions pass HC-1..HC-10. The engine scores all three and picks the highest:

```
Candidate regions: { West US 3, Central US, Canada Central }

Score(West US 3):    0.30 × 0.65 + 0.20 × 0.70 + 0.25 × 0.60 + 0.15 × 0.50 + 0.10 × 1.0 = 0.
Score(Central US):   0.30 × 0.40 + 0.20 × 0.55 + 0.25 × 0.75 + 0.15 × 0.45 + 0.10 × 1.0 = 0.
Score(Canada Central): 0.30 × 0.20 + 0.20 × 0.30 + 0.25 × 0.80 + 0.15 × 0.60 + 0.10 × 0.67 =

argmax → West US 3 (Prod anchor)
```

Subsequent CVAL and DR selections run the same scoring against the remaining eligible regions.

Sad Path — One Region Fails the Hard Constraint Gate Suppose Central US has a current NonProd quota usage so high that adding 3× E16ads_v5 (48 vCPU) would exceed the effective NonProd ceiling:

```
HC-7 check for Central US:
  nonprod_quota_used + (3 × 16) = nonprod_quota_used + 48 > effective_nonprod_ceiling
  → HC-7 FAIL → Central US excluded from scoring pool
```

The engine now scores only { West US 3, Canada Central }. It still picks the best of the remaining two — it does not fail the entire placement. Only if **all** Standard regions fail the hard constraint gate does the engine fall into the exception workflow.

3.6 EU Geography Walkthrough (2 Standard Regions)

Context: EU Geography has only two Standard Capacity regions: Switzerland North and Sweden Central. This is a structural constraint: with only two Standard regions, the engine cannot place Prod, CVAL, and DR in three separate regions.

How EU handles this — the co-location rule (PLC-010): one region is Prod, the other is both CVAL and DR together (co-located). CVAL and DR share the second region. This is by design, not a fallback.

The `CVALEarmarkRecord` prevents double-counting: CVAL capacity earmarked for DR activation cannot simultaneously be counted as live NonProd headroom.

Note: DR_NOT_OFFERED does not apply to EU. Both Switzerland North and Sweden Central are fully eligible for all three environment types.

Happy Path — Both Regions Eligible

```
Candidate regions: { Switzerland North, Sweden Central }

Step 1 (Prod):    argmax(PS_Prod) → say Switzerland North scores highest → Prod = Switzerlan
Step 2 (CVAL):    only Sweden Central remains → CVAL = Sweden Central (deterministic, no arg
```

```
Step 3 (DR):      same region as CVAL by PLC-010 co-location rule → DR = Sweden Central

CustomerSeedRecord:
  production_region = "Switzerland North"
  cval_region       = "Sweden Central"
  dr_region         = "Sweden Central"   ← co-located with CVAL
```

Sad Path — One Region Fails the Hard Constraint Gate If Sweden Central fails HC-3 (insufficient Prod quota headroom for this deployment):

```
HC-3 check for Sweden Central:
  prod_quota_headroom < requested_vm_count × vCPU
  → HC-3 FAIL → Sweden Central excluded from Prod scoring

Only remaining Prod candidate: Switzerland North
Prod = Switzerland North (deterministic - no argmax, single candidate)

CVAL + DR must go to Sweden Central (only remaining Standard region) regardless of HC failu
scoring. The engine re-evaluates HC for CVAL/NonProd context -
a region that fails Prod constraints
may still pass NonProd constraints because they check different quota groups/pools.
```

If **both** regions fail the gate for the same environment type, the engine has no eligible candidate and escalates to the exception workflow (Scenario 3).

3.7 Worked Example — E16ads_v5 (16 vCPU) at US Scale

Deployment: 3× E16ads_v5 = 48 vCPU for Prod in US Geography.

```
HC-3 Prod check for each candidate region:
  Required:  48 vCPU
  Region must have Prod_Group_headroom   48 vCPU

If West US 3 has Prod_Group_headroom = 64 vCPU → PASS (64   48)
If Central US has Prod_Group_headroom = 32 vCPU → FAIL (32 < 48) → excluded

Scoring runs on: { West US 3, Canada Central }
Winner: argmax of those two
```

---

4. Domain 2 — Quota Management

4.1 Concept

Quota is the Azure-imposed limit on how many vCPUs of a given VM family a subscription can deploy in a region. Even if physical capacity exists, deployment fails if the subscription has no quota left.

ACRME manages quota in a **single governed pool per region per VM family**. This pool covers Prod, NonProd/CVAL, and DR together. Inside the pool, ACRME uses **logical earmarks** to protect Prod and DR space — they are never available for NonProd to consume.

4.2 The Single Pool — Why One Pool?

Older designs used separate quota groups (one for Prod, one for NonProd+DR). The single-pool model was adopted because:

- **Flexibility:** all quota is in one place. If Prod needs more room, there is no inter-group transfer required — the engine just allocates from the shared pool.
- **Efficiency:** quota is expensive to obtain from Microsoft (quota increase requests take time and require justification). Pooling avoids the situation where one group is near-empty while another has idle headroom.
- **Emergency DR draw:** during an active DR event, the DR orchestrator draws directly from Pool_Headroom. No cross-group transfer; no Azure quota request needed for Tier 2 operations.

The two-group topology is retained as a narrow exception when Azure's own Quota Group API limits or a mandatory governance boundary make one pool impossible.

4.3 The Pool Formula — Translated

```
Pool_Limit(region) =
    Prod CRG quantity  × vCPU × 1.20   (Prod with 20% growth buffer)
  + NonProd CRG quantity × vCPU × 1.20 (NonProd with 20% growth buffer)
  + DR_Earmark_vCPU(region)             (space permanently reserved for DR standby)
  + emergency_transfer_headroom_vcpu    (30% of potential DR demand -
Tier 2 buffer)
```

What this means in plain English: the pool limit is sized to be big enough to hold everything at full scale, including a growth allowance, a permanent DR reserve, and a buffer for emergency DR capacity draws. When ACRME requests this quota from Microsoft, it is asking for the headroom to cover all of these uses without coming back for more quota frequently.

4.4 Logical Earmarks — Protecting Prod and DR Inside the Pool

Once the pool exists, the engine enforces two internal floors that NonProd can never consume:

Prod_Reserved_Floor: The minimum quota the Prod workloads need — their current vCPU usage plus the growth buffer above it. NonProd allocation is blocked if it would eat below this floor.

DR_Earmark_vCPU: The quota reserved for DR standby. This is calculated using the max-not-sum formula (see Domain 3 — Scenario 17): it is the capacity needed to absorb the *largest single source region* that could fail and send its customers here, not the sum of all potential sources.

Allocatable_NonProd is what is left over after both earmarks:

```
Allocatable_NonProd = Pool_Limit
                    - Prod_Reserved_Floor
                    - DR_Earmark_vCPU
                    - NonProd currently used
```

If this number hits zero, the engine blocks any further NonProd expansion in that region until either the pool limit is raised or Prod/DR demand decreases.

4.5 The DR Floor — Why It Exists and How It Is Enforced

The DR floor (`DR_Earmark_vCPU`) is a ring-fence: the engine promises that there will always be enough quota to activate DR for the largest protected source region. Without it, NonProd growth could quietly eat the DR quota until a real DR event arrives — at which point the DR activation fails.

Azure itself does not natively enforce intra-pool sub-reservations. The engine enforces the DR floor by arithmetic at command time (when NonProd expansion is requested) and by a separate detector that continuously recomputes the floor from authoritative assignment data. If the detector disagrees with the command-time calculation, automatic NonProd expansion is disabled until the state is reconciled.

4.6 US Geography Walkthrough — Quota Sizing

Context: 3× E16ads_v5 (48 vCPU) for Prod, 3× E8ads_v5 (24 vCPU) for NonProd, in US Geography (3 Standard regions: West US 3, Central US, Canada Central). DR requirement sized by max-not-sum.

Worked example for one region (West US 3):

```
Inputs:
  Prod CRG quantity     = 48 vCPU  (3× E16ads_v5 × 16 vCPU)
  NonProd CRG quantity  = 24 vCPU  (3× E8ads_v5  × 8 vCPU)
  DR_Earmark_vCPU       = 48 vCPU  (sized for the largest source -
see Domain 3)
  emergency_transfer_headroom = 24 × 0.30 = 7.2 → ceil → 8 vCPU

Pool_Limit = (48 × 1.20) + (24 × 1.20) + 48 + 8
           = 57.6 + 28.8 + 48 + 8
           = 142.4 → request 143 vCPU from Microsoft

Prod_Reserved_Floor   = 48 + (48 × 0.20) = 57.6 vCPU
```

```
DR_Earmark_vCPU       = 48 vCPU
NonProd currently used = 24 vCPU (all in use)

Allocatable_NonProd = 143 - 57.6 - 48 - 24 = 13.4 vCPU remaining
```

Happy Path — NonProd Expansion Request Within Allocatable Headroom A team requests 1 additional E8ads_v5 (8 vCPU) for NonProd:

```
HC-7 check: NonProd used + 8 = 24 + 8 = 32 > Allocatable_NonProd ceiling?
  Allocatable_NonProd ceiling = Pool_Limit - Prod_Reserved_Floor - DR_Earmark_vCPU
                              = 143 - 57.6 - 48 = 37.4 vCPU
  32   37.4 → PASS
NonProd expansion approved.
```

Sad Path — NonProd Expansion Would Violate DR Floor A team requests 4× E8ads_v5 (32 vCPU) for NonProd at once:

```
HC-7 check: NonProd used + 32 = 24 + 32 = 56 > effective NonProd ceiling of 37.4 vCPU
  → HC-7 FAIL
  → Emit: DRFloorViolationDetected (Critical severity)
  → Block expansion
  → Alert: "NonProd expansion denied - DR floor integrity violated"
```

The team must request a quota increase from Microsoft (raising Pool_Limit) before this deployment can proceed.

4.7 EU Geography Walkthrough — Quota with Co-location

In EU, CVAL and DR co-locate in the second Standard region (say Sweden Central). The pool for Sweden Central must cover both NonProd and DR earmarks:

```
Prod region:        Switzerland North
CVAL + DR region:   Sweden Central

Sweden Central pool covers:
  NonProd CRG quantity  = 24 vCPU
  DR_Earmark_vCPU       = sized for DR requirement
  (Prod_Reserved_Floor applies only in Switzerland North)

CVALEarmarkRecord enforces: CVAL capacity used for DR activation is NOT
  simultaneously counted as Allocatable_NonProd headroom.
```

This prevents a double-count: CVAL VMs that are earmarked to be shut down and replaced by DR standby cannot be claimed as "available NonProd headroom" while they are still running.

4.8 Quota Scoring (Scenario 10) — The Component

The component of the placement score (weight 0.20) reflects the *current* quota position. It is dispatched by environment type:

```
Quota_Score_Prod(r)    = prod quota headroom ÷ prod quota limit
Quota_Score_NonProd(r) = NonProd headroom ÷ effective NonProd ceiling
Quota_Score_DR(r)      = DR headroom ÷ DR floor vCPU
```

Important nuance on DR: a DR score of 1.0 means maximum expansion room (DR CRG is at zero — nothing is reserved yet, so the ceiling has not been eaten). It does not mean "quota is fully consumed." A DR score of 0 means the DR earmark is fully used — the region cannot take more DR load.

---

5. Domain 3 — Capacity Management

5.1 Concept

Quota (Domain 2) answers whether the *subscription limit* allows a deployment. Capacity management (Domain 3) answers whether *Azure has actually set aside physical hardware* for those VMs.

This is managed through **Capacity Reservation Groups (CRGs)**. A CRG tells Azure: "I want you to guarantee that a certain number of VMs of this SKU in this region/zone will always be available to me." Azure marks that physical hardware as reserved; other customers cannot take it.

ACRME's capacity management job is to keep the CRG quantity correctly sized — not too high (wastes money) and not too low (exposes customers to deployment failures).

5.2 The Reconciliation Floor — The Minimum the CRG Must Reserve

The most fundamental formula is:

```
Target Reserved Capacity = Allocated VM Count + Configured Buffer
```

Allocated VM Count is the number of VMs currently in `running` or `allocated` state consuming compute in this region/CRG.

Configured Buffer is the policy-defined headroom above current demand. It is tunable per product/region/environment/SKU — there is no single hard-coded number.

What this means in plain English: at minimum, the CRG must reserve enough physical capacity to cover every running VM *plus* a buffer for expected near-term growth. The buffer is there because acquiring new reservation capacity takes time — ACRME stays ahead of demand.

Important clarification — deallocated VMs (CAP-004): VMs that are associated with a CRG but currently deallocated (stopped/not running) do **not** force the CRG to keep a reservation for them. They are reported separately so teams understand restart risk, but they do not add to the Target. The risk of a failed restart is acknowledged — it is a cost vs. reservation trade-off.

5.3 Reservation Headroom and Deficit

```
Reservation Headroom = Reserved Quantity - Allocated VM Count
Reservation Deficit  = max(0, Target Reserved Capacity - Reserved Quantity)
```

Headroom tells the operator how much spare capacity is immediately available inside the existing reservation. **Deficit** tells the operator how far short the reservation is of the minimum target. A deficit of zero means the CRG is at or above the floor. A positive deficit means the CRG is undersized and needs to grow.

5.4 The Auto-Increase Trigger — When ACRME Raises a Capacity Request

ACRME does not wait for a deficit to appear before acting. It watches utilisation in real time and triggers a capacity increase request when utilisation hits a threshold:

Environment	Utilisation threshold	Meaning
Prod	20%	Trigger when 20% of reserved Prod capacity is consumed
NonProd	20%	Trigger when 20% of reserved NonProd capacity is consumed
DR	35%	Trigger when 35% of reserved DR capacity is consumed (DR bootstrap can run leaner)

Debounce cooldown: after any trigger fires, the engine waits 30 minutes before it can fire again for the same region + CRG type. This prevents repeated requests from a transient spike.

Why the DR threshold is higher (35% vs 20%): DR reservations are intentionally sized lean — the min-bootstrap model means DR starts small and scales on demand. A higher threshold accepts more risk in exchange for lower cost during steady state.

5.5 The Forecast Formula — Proactive Growth

Alongside the continuous reconciliation floor, ACRME uses an advisory forecast for longer-horizon sizing:

```
Forecast_Quantity = ceil( Forecast_Peak × (1 + Growth_Buffer) + DR_Buffer )
```

- **Forecast_Peak** is the predicted peak VM demand over the chosen horizon (30, 60, or 90 days).
- **Growth_Buffer** is a policy percentage for forecast uncertainty.
- **DR_Buffer** is the extra units required by DR recovery policy.

This is advisory — recommendations from the forecast are reviewed before acting. The horizon and buffer values are stored in PlacementPolicy, not hardcoded.

Lead-time alerting: when forecast demand is projected to reach 80% of the quota limit within 14 days, the engine emits `ForecastApproachingQuotaLimit`. This gives the platform team 14 days to request a quota increase from Microsoft before the deployment pipeline hits a wall.

5.6 DR Sizing — Why Max, Not Sum (Scenario 17)

This is the biggest cost-driver decision in ACRME. Before v2.1, DR standby was sized as a **percentage of production capacity** (30–40%). This was found to over-provision by $1.5M–$5M/year at platform scale.

The replacement formula is:

```
Destination_DR_Requirement(destination_region) =
    MAX over all source regions that send DR customers to this destination (
        the quantity of VMs/vCPUs from that source that must failover here
    )
```

Why MAX and not SUM? The design assumes **only one production region fails at a time** (the single-failure assumption, DR-001). Because failures are mutually exclusive, the destination never needs to host simultaneous failovers from multiple sources. It only ever absorbs the *largest* one.

Example from Requirements Appendix D:

```
Sweden Central hosts DR standby for:
  → Switzerland North customers: 120 cores
  → (hypothetically) a third EU region: 80 cores

SUM sizing (old): 120 + 80 = 200 cores required
MAX sizing (new): max(120, 80) = 120 cores required
Saving: 80 cores (40%) removed with no loss of protection
```

If the single-failure assumption is ever violated (two regions fail simultaneously), the standby is overcommitted. The Overcommit_Ratio quantifies this exposure and is required input for the POC-011 risk sign-off that gates production reliance on max-not-sum.

A **SUM override (C-11)** is available per-scope in PlacementPolicy for customers or geographies that contractually require protection against concurrent regional failures.

5.7 Happy Path — Steady-State Capacity Management

All conditions normal for West US 3, Prod CRG, E16ads_v5:

```
Allocated VM Count     = 3  (the 3 production VMs are running)
Configured Buffer      = 2  (policy: always keep 2 extra slots ready)
Target Reserved        = 5

Reserved Quantity      = 5
Reservation Headroom   = 5 - 3 = 2
Reservation Deficit    = max(0, 5 - 5) = 0

Utilisation = 3 / 5 = 60%
20% threshold not breached (utilisation is above threshold in the wrong direction -
this
  would actually be concerning, but the threshold fires when utilisation hits the threshold
  from below, meaning the CRG is filling up)
```

Wait — clarifying the threshold direction: the auto-increase trigger fires when utilisation of the CRG **exceeds** the threshold, i.e., the CRG is getting full. At 60% utilisation on a Prod CRG (threshold 20%), the CRG is already well above threshold — the engine would have already raised a capacity increase request earlier, when utilisation first hit 20%.

Revised happy path:

```
Allocated VM Count     = 1 of 5 reserved slots used (20% utilisation)
Utilisation = 1 / 5 = 20%
Threshold   = 20%  → trigger fires → CapacityIncreaseRequest raised
```

The request is raised, debounce cooldown starts (30 min), operator approval obtained (Phase 1), new CRG quantity updated.

5.8 Sad Path — Azure Cannot Supply the Requested Capacity

After ACRME raises a capacity increase request and submits it to Azure, Azure responds that it cannot supply the requested SKU/zone at this time:

```
Azure response: CAPACITY_UNAVAILABLE for E16ads_v5 in West US 3, Zone 2
ACRME action:
  1. Hold current CRG quantity (do not reduce below current level).
  2. Raise alert: "Capacity unavailable - reservation increase failed for West US 3 / E16ads
  3. Expose buffer deficit in the observability dashboard.
```

```
4. Do NOT delete or reduce the reservation object (CAP-009/010).
5. Retry according to the reconciliation cadence (target 6-minute cycle).
6. Escalate to emergency transfer tiers if the deficit is critical and a DR event is activ
```

The deployment readiness gate (Scenario 19) returns `CAPACITY_UNAVAILABLE` for any new deployment request that depends on this CRG until the situation resolves.

5.9 Standby Activation During DR (Scenario 18)

When a DR event is declared and the engine enters `DR_EVENT_ACTIVE`:

1. Standby VMs (currently `associated` with the destination CRG but not running) transition to `allocated` (active, consuming capacity).
2. This happens in **priority waves**: Wave 0 (platform control planes first), then Wave 1 (highest-priority customers), then subsequent waves in business-priority order.
3. Each wave goes through up to 7 stages of capacity acquisition:
    - Stages 1–2: use pre-staged bootstrap headroom and existing reservation — zero Azure wait time.
    - Stage 3: shut down eligible CVAL workloads (after authorisation) to free their slots.
    - Stages 4–6: share reservations, draw pooled quota, request additional Azure capacity.
    - Stage 7: report unrecoverable gaps.

The design ensures P0/P-1 customers begin recovery without waiting on Azure deallocation APIs — which are throttled during regional failure events.

---

6. Scoring Weight Invariant

6.1 The Rule

The five placement scoring weights must always sum to exactly 1.0:

```
 +   +   +   +   = 1.0
0.30 + 0.20 + 0.25 + 0.15 + 0.10 = 1.00
```

This is not a convention. It is a mathematical requirement for the scores to be meaningful.

6.2 Why 1.0 Is Required

Every component is clamped to [0, 1]. When weights sum to exactly 1.0:

- The **best possible region** (all components = 1.0) scores exactly **1.0**.
- The **worst possible region** (all components = 0.0) scores exactly **0.0**.
- Every real region scores **somewhere between 0 and 1**.

- Scores from different regions are **directly comparable**: 0.75 is always better than 0.60, in every run, under every policy version.

6.3 What Breaks When the Weights Don't Sum to 1.0

If weights sum to more than 1.0 (e.g., 1.10):

The best possible score inflates above 1.0. A region with all perfect components scores 1.10 instead of 1.0. Any downstream threshold calibrated against a [0, 1] range — for example, "do not place in any region scoring below 0.60" — is now calibrated incorrectly. A mediocre region may clear the threshold simply because scores are inflated.

If weights sum to less than 1.0 (e.g., 0.90):

All scores are uniformly compressed into [0, 0.90]. A region with all perfect components scores 0.90 instead of 1.0. A region that should score 0.67 now scores 0.60. Threshold gates calibrated against [0, 1] now behave incorrectly in the other direction — some deployments are incorrectly blocked.

Cross-policy comparisons also break: if one version had weights summing to 1.0 and the next has them summing to 0.90, a score of 0.65 means something different under each version. Historical score trend data becomes uninterpretable.

6.4 Tuning the Weights — The Pie Rule

Think of the five weights as slices of a pie. The total pie is always exactly 100%. If one slice grows, the others must shrink by the same total amount.

Example — increasing headroom importance:

```
Before:  =0.30,  =0.20,  =0.25,  =0.15,  =0.10  → sum = 1.00
After:   =0.40,  =0.15,  =0.25,  =0.10,  =0.10  → sum = 1.00  (took 0.05 from  , 0.05 from  )
Wrong:   =0.40,  =0.20,  =0.25,  =0.15,  =0.10  → sum = 1.10  (forgot to reduce)
```

6.5 Validation Requirement — The Policy Load Gate

The Config/Scope-File Service **must** validate the weight sum every time a new PlacementPolicy version is loaded. This validation must occur **before** the policy version is published to the placement engine. A policy that fails this check must be rejected with a recorded configuration error; the placement engine must never receive a policy with an invalid weight sum.

See Section 6 of the Calculation Logic Reference and Section 8.2 of the Technical Design Document for the formal validation rule specification.

---

7. How the Three Domains Connect

The three domains are not independent. They feed into each other:

```
Region Selection → Quota Management → Capacity Management
       ↑                  ↑                    ↑
  Uses quota scores   Uses CRG quantities   Uses placement output
  as   component     to size pool limit    to set CRG targets
```

Flow for a new customer deployment:

1. **Region Selection** scores all candidate regions and picks the best Prod anchor. The quota score (component) uses current quota headroom from Domain 2. The capacity/headroom score (component) uses current CRG free slots from Domain 3.

2. **Quota Management** checks that the subscription in the selected region has enough quota limit to accept the new deployment (Scenario 19, gate 7). If not, the deployment is blocked with `QUOTA_DEFICIT` until a quota increase request to Microsoft is approved.

3. **Capacity Management** checks that a Capacity Reservation exists and has enough quantity for the deployment (Scenario 19, gate 6). If not, the deployment is blocked with `RESERVATION_DEFICIT` until the CRG is expanded.

Only when all three domains return green does the readiness gate emit `READY` and allow the deployment to proceed.

Flow for a DR event:

1. The engine transitions to `DR_EVENT_ACTIVE`.
2. The DR Orchestrator consults the SourceDestinationDRIndex to identify which standby VMs need to be activated and where.
3. **Capacity Management** begins staged activation (Scenario 18): bootstrap headroom first, then CRG expansion if needed, then CVAL release, then quota draw.
4. **Quota Management** confirms the shared pool has headroom for the DR draw (single-pool model: no cross-group transfer needed — the DR earmark is already inside the pool).
5. **Region Selection** is not re-run — the Customer Seed Record is authoritative. Failback restores the original seed without re-derivation.

---

8. Quick Reference — Key Numbers

Parameter	Value	Domain
(headroom/capacity weight)	0.30	Region Selection
(quota headroom weight)	0.20	Region Selection

Parameter	Value	Domain
(distribution fairness weight)	0.25	Region Selection
(DR readiness / overflow weight)	0.15	Region Selection
(zone diversity weight)	0.10	Region Selection
Weight sum (must equal)	**1.00**	Region Selection
Weight sum validation tolerance	0.001	Region Selection
Prod growth buffer	20%	Quota Management
NonProd growth buffer	20%	Quota Management
Emergency transfer headroom	30% of potential DR demand	Quota Management
Min Prod quota headroom	20 vCPU	Quota Management
Min NonProd quota headroom	20 vCPU	Quota Management
Min DR headroom	16 vCPU	Quota Management
Prod auto-increase threshold	20% utilisation	Capacity Management
NonProd auto-increase threshold	20% utilisation	Capacity Management
DR auto-increase threshold	35% utilisation	Capacity Management
Auto-increase debounce cooldown	30 min per region + CRG type	Capacity Management
Forecast horizons	30 / 60 / 90 days	Capacity Management
Quota alert lead time	14 days at 80% of limit	Capacity Management
Reconciliation loop target	6 minutes (configurable)	Capacity Management
US Standard Capacity Regions	West US 3 · Central US · Canada Central	All domains
EU Standard Capacity Regions	Switzerland North · Sweden Central	All domains
Middle East Standard Capacity Regions	UAE North · Saudi Arabia Central	All domains
Middle East DR status	`DR_NOT_OFFERED` (DEC-001 pending)	Region Selection
EU CVAL + DR co-location	Mandatory (PLC-010) — 2-region geography	Region Selection

---

Document version 1.0 — 2 September 2026. Source: ACRME Q&A design *walkthrough session, Calculation Logic Reference v2.2, Requirements Baseline v2.2. Next review: when Calculation Logic Reference is next revised or when POC-001/POC-011 results are available.*